The diagram is a reasonable starting point, but it has a few issues that affect accuracy, readability and how well it communicates the design. Below is an improved version followed by an explanation of each change.

Improved PlantUML Code

```
@startuml
skinparam classAttributeIconSize 0
skinparam backgroundColor #F9F9F9
skinparam classFontStyle bold
skinparam packageStyle rectangle

package "UI Layer" {
  class MenuUI {
    + displayMenu(): void
    + getChoice(): int
    + handleChoice(choice: int): void
    + displayStatus(machine: VoleMachine, memory: Memory): void
  }
}

package "Core Emulator" {
  class VoleMachine {
    - pc: int
    - ir: int
    - registers: int[16]
    - isRunning: boolean
    + powerOn(): void
    + powerOff(): void
    + runProgram(): void
    + executeStep(): void
  }

  class ControlUnit {
    + fetch(pc: int, memory: Memory): int
    + decode(instruction: int): void
  }

  class ALU {
    + executeALUOperation(opcode: int, r: int, val1: int, val2: int):
int
  }

  class Memory {
    - size: int = 256
    - cells: byte[256]
```

```
    + read(address: int): byte
    + write(address: int, value: byte): void
    + loadProgram(program: byte[]): void
    + dumpMemory(): void
  }
}

' Composition: VoleMachine owns its parts
VoleMachine "1" *-- "1" Memory : contains >
VoleMachine "1" *-- "1" ControlUnit : contains >
VoleMachine "1" *-- "1" ALU : contains >

' Dependencies: UI depends on the emulator
MenuUI ..> VoleMachine : controls >
MenuUI ..> Memory : inspects/edits >

note right of VoleMachine
  Registers: 16 general-purpose
  registers (R0-RF), each 8 bits.
  PC and IR are 8-bit in VOLE.
end note

note right of Memory
  256 cells of 1 byte each
  (addresses 0x00-0xFF).
end note

@enduml
```

What Was Improved and Why

1. Added packages to group related classes

The original diagram placed all classes on one flat canvas. Grouping them into **UI Layer** and **Core Emulator** packages makes the separation of concerns visible at a glance. The user interface depends on the emulator, not the other way round, and the package boundary communicates that clearly.

2. Fixed the relationship labels

The original labels `manages >`, `uses >` and `controls >` are placed on the association lines, but the `>` symbol is a directional arrow hint that PlantUML does not treat as

part of the label syntax in a meaningful way. Using `contains >` for composition better reflects that the machine owns these parts, and the arrow in the label reads naturally with the composition diamond. The labels are now consistent with the composition semantics.

3. Fixed the composition semantics for ControlUnit and ALU

The original used `uses` for both ControlUnit and ALU, which is inconsistent with the solid diamond (composition) arrow. A composition relationship means the parent owns the child's lifecycle, so `contains` is the accurate word. The original wording suggested a looser dependency.

4. Added an explicit default to Memory size

The attribute `size: int` and the fixed array `cells: byte[256]` are redundant and can drift out of sync. Setting `size: int = 256` makes the intent clear and signals that the value is a constant for this architecture.

5. Added notes explaining architectural constants

In the VOLE machine (the simplified architecture used in many Computer Organisation courses), the program counter and instruction register are 8 bits wide, registers are 8-bit, and memory has 256 one-byte cells. The original used `int` for these, which is technically looser than the real design. Notes document these constraints without cluttering the class boxes.

6. Added bold class names

Setting `classFontStyle bold` makes class names stand out, which improves scanning in larger diagrams.

7. Kept the original styling

The `classAttributeIconSize 0` and background colour were retained, so the look stays consistent with the original.

Summary

The improved version groups classes logically, uses clearer and more accurate relationship wording, documents the architectural constants that the `int` types hide, and presents the structure in a way that is easier to read and maintain.